

Gymnázium J. K. Tyla



Agentické inženýrství ve vývoji softwaru

Návrh a implementace DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnáziu J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Práce představuje agentickou umělou inteligenci jako poslední krok přechodu od konverzační asistence k delegovanému vývoji a vymezuje pojem agentického inženýrství. Ukazuje, že jádrem této disciplíny není schopnost jazykového modelu, ale návrh systému, v němž model pracuje, a že pojem software factory, zavedený již v roce 1968, zažívá právě prostřednictvím agentů znovu. Výzkumná otázka zní, které principy musí agentický systém splnit, aby vykonával inženýrskou práci. Hypotéza, kterou práce v praktické části ověřuje, na ni odpovídá: praktická autonomie je vlastností návrhu systému, který práci řídí, a nikoli vlastností modelu, který v něm pracuje. Praktická část popisuje DarkFactory, záměrně minimální produkční softwareovou továrnu, v níž je GitHub vývojovým prostředím i trvalým stavem a pythonovský řadič spolu s produkčními harnessy realizuje interpretaci, plánování, implementaci, revizi a integraci změn. Strukturu, která práci řídí, navrhuje autor sám. Úsudek o tom, co je v jednotlivém kroku správné, však přebírá od smyčky, kterou nevlastní — a právě v tomto rozdílu leží hranice, za kterou teprve vlastní harness pomůže.

Klíčová slova: coding agents; software factory; AI; Agentic Engineering; harness

Abstract

This thesis presents agentic artificial intelligence as the latest step in the transition from conversational assistance to delegated development, and establishes the term agentic engineering. It argues that the core of the discipline is not the capability of a language model but the design of the system in which the model operates, and that software factory, a concept introduced in 1968, is being revived precisely through agents. The research question asks which principles an agentic system must satisfy in order to perform engineering work, and the hypothesis tested in the practical part answers it: practical autonomy is a property of the design of the system that governs the work, not of the model operating inside it. The practical part describes DarkFactory, a deliberately minimal production software factory in which GitHub is the development environment and the durable state, GitHub Actions the execution environment, a container the isolation layer, and a Python runner working with production harnesses performs intake, planning, implementation, review and integration. The structure that governs the work is the author's own; the judgement of what is correct in any single step is taken over from a loop he does not own — and beyond that difference lies the boundary that only a harness of one's own helps to cross.

Keywords: coding agents; software factory; AI; Agentic Engineering; harness

Obsah

Prohlášení	2
1 Úvod	5
1.1 Motivace: Vývoj a adopce generativní AI	5
1.2 Cíl, výzkumná otázka, hypotéza a vymezení	8
1.3 Terminologie	8
2 Teoretická část	9
2.1 Agent: Co to je a jak funguje	9
2.1.1 Jazykový model (LLM)	9
2.1.2 Smyčka agenta (Agent Loop)	10
2.1.3 Nástroje, oprávnění a paměť (Tools, Permissions & Memory)	11
2.1.4 Kontextové okno a kompakce	12
2.2 Agentické inženýrství	14
2.2.1 Zadání a plán (Prompt & Plan)	14
2.2.2 Orchestrace a lidská integrace (Orchestration & HITL)	16
2.3 Softwareová továrna	19
3 Praktická část	21
3.1 Metodika	21
3.2 Architektura produkčního běhu	22
3.3 Interpretace a plánování požadavku	24
3.4 Implementace a automatická revize	26
3.5 Zpětná vazba, schválení a úklid	27
4 Závěr	28
4.1 Zjištění	28
4.2 Diskuse	29
4.3 Shrnutí	32
Seznam zdrojů	34
Seznam součástí textu	38

1 Úvod

1.1 Motivace: Vývoj a adopce generativní AI

Nástroje založené na jazykových modelech prošli v krátké době velkým rozvojem. Nejprve doplňovaly kód v editoru (1).

A screenshot of a code editor interface. At the top, it says 'JS test.js 1' with a small circle icon. Below that, a prompt 'JS test.js > calculateDaysBetweenDates' is shown. The main area displays a JavaScript function definition: '1 function calculateDaysBetweenDates(begin, end) {' followed by three lines of code: 'var beginDate = new Date(begin);', 'var endDate = new Date(end);', and 'var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));'. The function ends with 'return days;' and a closing brace '}'. A line number '2' is visible at the bottom left of the code block.

```
JS test.js 1 ●  
  
JS test.js > calculateDaysBetweenDates  
1 function calculateDaysBetweenDates(begin, end) {  
    var beginDate = new Date(begin);  
    var endDate = new Date(end);  
    var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));  
    return days;  
}  
2
```

Obrázek 1: Doplňování kódu přímo v editoru: model navrhuje pokračování řádku, které člověk přijme nebo odmítne (1).

Potom přišly konverzační chatboty (2), v nichž model sestavuje odpověď, ale nástroje mu zpravidla nebyly k dispozici, takže i nadále všechno provedl uživatel.

P

A 59-year-old female comes to your office for routine examination. On a panoramic radiograph, you noticed some incidental findings. Please provide your diagnosis based on the radiograph below.

1. Florid cemento-osseous dysplasia
2. Secondary hyperparathyroidism
3. Fibrous dysplasia
4. Hypercementosis



As an AI language model, I'm unable to directly view or interpret images. However, I can provide you with a brief description of the conditions you mentioned to help you understand them better.

1. Florid cemento-osseous dysplasia: This is a benign condition that affects the jawbones, specifically the cementum and surrounding bone. It typically appears as multiple radiopaque masses in the jawbones and is often asymptomatic.
2. Secondary hyperparathyroidism: This condition occurs due to chronic kidney disease or vitamin D deficiency, leading to abnormal parathyroid gland function. It can result in increased bone resorption and subsequent radiographic changes.
3. Fibrous dysplasia: Fibrous dysplasia is a bone disorder where normal bone is replaced by fibrous tissue. It can present as an incidental finding on radiographs, showing a radiolucent and ground-glass appearance in affected areas.
4. Hypercementosis: Hypercementosis is a condition characterized by excessive deposition of cementum around the roots of teeth. It can appear as a radiopaque enlargement around the roots of affected teeth.

Without visual examination and additional information, it is challenging to provide an accurate diagnosis based solely on a radiograph. It is recommended to consult with a qualified dental or medical professional who can evaluate the patient's clinical history, symptoms, and perform a physical examination to provide an accurate diagnosis.

Obrázek 2: Chatbot odpovídá textem a práci nechává člověku: místo aby snímek posoudil, přizná, že obrázky neumí, a je to uživatel, kdo musí dojít k závěru. Screenshot z bezplatné verze ChatGPT, převzato z (3).

Až třetí stupeň, *agenti* (4; 5; 6), dostal přístup k souborům, příkazům a běhovému prostředí — a tím se poprvé změnilo kdo pracuje.



Obrázek 3: Odhad rozdělení uživatelů generativní AI podle typu (7).

S rozvojem nástrojů roste i jejich adopce. Rozsah veřejného použití samotných agentů přitom stále úzký. Jeden ze zveřejněných odhadů klade počet uživatelů chatbotů na 28 % populace a pravidelné užití *agentů* na 0,36 % světové populace (7) Obrázek 3. Přesto je zřejmé, že se jedná o technologii, která se rychle šíří a mění způsob práce, proto v této práci chci ukázat čeho jsou plnohodnotné systémy schopné.

1.2 Cíl, výzkumná otázka, hypotéza a vymezení

Výzkumná otázka práce zní: *Které principy musí agentický systém splnit, aby spolehlivě vykonával inženýrskou práci?* Inženýrskou prací se zde rozumí změna repozitáře, kterou může jiný člověk než její autor přezkoumat a sloučit, aniž by musel agenta na cokoli ptát; v tomto smyslu je měřena schopnost systému a v tomto smyslu je brána člověka nezaměnitelnou.

Práce předpokládá tuto hypotézu:

Praktická autonomie je vlastností návrhu systému, který práci řídí, a nikoli vlastností modelu, který v něm pracuje.

Hypotéza se ověřuje v praktické části a z jejího výsledku práce vychází.

Praktická část popisuje DarkFactory, záměrně minimální produkční pipeline pro AI-asistovaný softwarový vývoj (8). Její běh, brány a uložený stav tvoří harness této práce; jako vykonávací engine používá produkční harnessy, protože to je pro popsanou míru detailu nejjednodušší.

Rozsah práce je záměrně úzký a odpovídá jedné revizi repozitáře: popisuje počáteční implementaci, v níž je agentní smyčka (*Agent Loop*) provedena cizím nástrojem. Její rozsah slouží jako základ, na němž navazuje práce následující.

1.3 Terminologie

Překlad anglických výrazů v této práci sleduje jedno pravidlo. Co anglicky nazývá **agent**, je česky **agentní** — agent, agentní krok, agentní smyčka. Co anglicky nazývá **agentic**, je česky **agentické** — agentický systém, agentické inženýrství. Podstatné jméno **agent** se tedy nepřekládá, protože označuje samotného jednajícího, zatímco přípona **-ic** znamená majícího schopnost jednat, nikoli jednajícího (9). Podle téhož pravidla je *Agent Loop* česky *agentní smyčka*, protože jde o smyčku agenta, a nikoli o vlastnost systému.

2 Teoretická část

2.1 Agent: Co to je a jak funguje

Agent je model plus harness (10): model navrhuje a harness mu dopředuje kontext, nástroje a pravidla, jejichž prostřednictvím se návrh mění v čin. Celý systém tvoří několik vrstev — rozhraní, smyčka agenta, oprávnění, nástroje, uložený stav a prostředí, v němž běží — a model je jen jedna z nich. Spolehlivost agenta proto neurčuje samotný model, ale skladba těch vrstev; a čím jsou modely schopnější, tím větší část práce přebírá tato skladba, ne model.

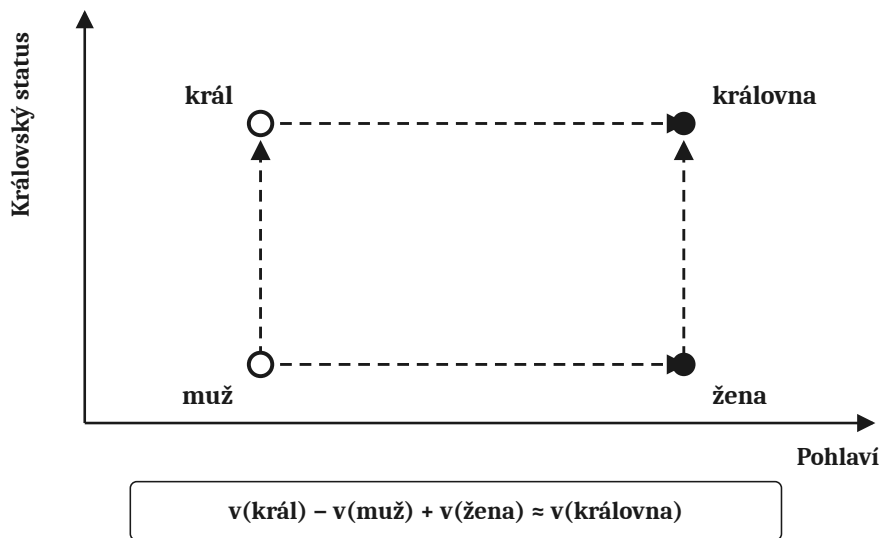
2.1.1 Jazykový model (LLM)

Současný jazykový model stojí na architektuře **Transformer**, kterou představil Google v roce 2017 v jejich nyní proslulé práci „*Attention Is All You Need*“ (11). Jejím základem je **attention**: ke každému tokenu připočítá vážený součet hodnot všech ostatních, takže jeho reprezentace nese informaci z celého kontextu a vzdálenost mezi pozicemi není pevně daná jejich pořadím. Proto lze kontext zpracovat najednou, což je podmínkou dnešních nároků na délku a složitost konverzace; mechanismus je používán i v pozdějších generacích modelů (12).

Jazykový model je sám o sobě jen funkce. Na základě toho, co dostane v kontextu, vypočítá rozložení pravděpodobností nad následujícím tokenem a jeden z nich vybere. Mezi voláními si nic nepamatuje, nezná nic mimo text, který mu byl předložen, a nemá přístup k souborům, příkazům ani stavu práce. Všechno, co o zadání ví, je v kontextu, a jindy se to do něj nedostane.

Při **inferenci** tedy model zpracuje obsah kontextového okna a vytváří výstupní tokeny jeden za druhým, přičemž každý nový token se stane součástí kontextu pro další krok. To je celá jeho schopnost: převést kontext na posloupnost tokenů. Stav mezi kroky, nástroje, ukládání výsledků a oprávnění musí přijít zvenčí — a to je právě to, co doplňuje harness.

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy v prostoru vektorů. Známým příkladem je vztah mezi vektory slov král, královna, muž a žena (13). Tento vztah schematicky znázorňuje Obrázek 4.



Obrázek 4: Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov *král*, *královna*, *muž* a *žena* (13).

2.1.2 Smyčka agenta (Agent Loop)

Základním mechanismem agentického systému je **agentní smyčka**: konkrétní implementace vzoru **ReAct** (*Reasoning and Acting*) (14) v daném harnessu. Jeden cyklus má pět kroků:

- sestavit kontext: systémové pokyny, dosavadní přepis, stav prostředí;
- nechat model odpovědět;
- je-li v odpovědi požadavek na nástroj, provést jej;
- zapsat výsledek nástroje jako pozorování;
- vrátit se na první krok.

Model přitom v každém cyklu nejprve zdůvodní, co udělá, a teprve pak žádá o nástroj (15). Harness akci provede mimo model a výsledek mu vrátí; ten se připojí k přepisu a cyklus běží dál. Smyčka končí teprve tehdy, když model místo požadavku na nástroj vydá závěrečnou odpověď. V prosté posloupnosti promptů by model nemohl poznat, zda předchozí krok uspěl; průběh shrnuje Obrázek 5.



Obrázek 5: Vzor ReAct: myšlenka, akce a pozorování se střídají a každý z nich se připojí do kontextu, takže smyčka pokračuje na tom, co už model viděl. Končí odpovědí místo dalšího požadavku na nástroj (14).

2.1.3 Nástroje, oprávnění a paměť (Tools, Permissions & Memory)

Zadání určuje, co má agent udělat; co mu je dovoleno udělat, určuje harness. Jde o dvě roviny, které se dají měnit nezávisle, a odpovídají běžnému výčtu součástí harnessu (10):

- **Nástroje** umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy. Než se nástroj spustí, harness jeho vstup ověří proti předepsanému schématu, takže chybný požadavek se neprovede.
- **Skills** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy (16).
- **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit kontrolu (17).
- **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient–server (18).

Každá akce ještě předtím projde oprávněním, které pro ni platí: provést automaticky, zeptat se člověka, nebo zakázat. Ani když model navrhne něco nebezpečného, dostane se to k provedení jen tehdy, když to oprávnění dovolí.

Instrukce pro agenta leží v repozitáři, standardně v souboru `AGENTS.md` (19) (u Anthropic v `CLAUDE.md`). Takový soubor může existovat v několika úrovních — v kořeni projektu,

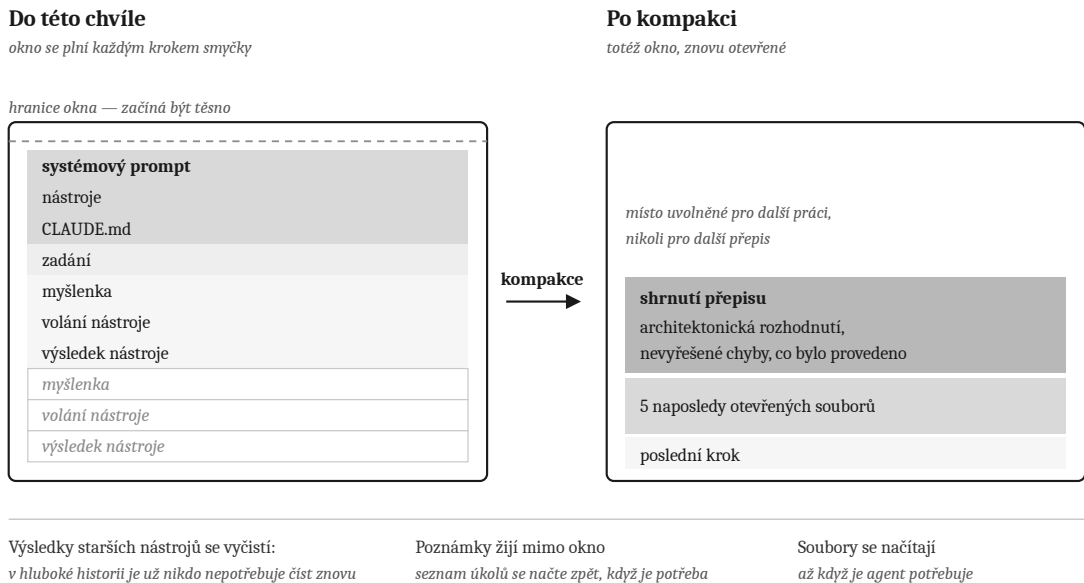
ve složce i v jednotlivých podadresářích — a harness si načte jen ten, který se týká právě otevřených souborů. Skilly, scripty a hooky leží pod složkou `.agents/`.

2.1.4 Kontextové okno a kompakce

Kontextové okno zahrnuje pracovní kontext jednoho volání modelu: instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho kapacita sama o sobě nezaručuje, že model všechny podstatné informace správně využije — úspěšnost jejich vybavení závisí také na umístění a s rostoucí délkou vstupu klesá (20). Toto zhoršování práce s nahromaděným kontextem se označuje jako **context rot** (21).

Kompakce (*compaction*) po překročení limitu nahradí starší průběh souhrnem rozhodnutí a výsledků, takže do dalšího volání není nutné vkládat celý přepis (21).

V konkrétní implementaci se ale celá trajektorie skládá do jednoho okna, a náklady začínají tam. Výsledky starších nástrojů se z přepisu vyčistí, protože v hluboké historii je už nikdo nepotřebuje číst znovu; rozhodující je, co zůstane v souhrnu. Živé zpracování jednoho běhu v Claude Code shrnuje Obrázek 6.



Obrázek 6: Kontextové okno v Claude Code během jednoho běhu: systémový prompt, nástroje a zadání zůstávají, zatímco myšlenky, volání nástrojů a jejich výsledky se vrší až k hranici okna. Po kompakci zůstane souhrn rozhodnutí, poslední krok a pět naposledy otevřených souborů. Vedle toho běží tři věci: starší výsledky nástrojů se čistí, poznámky žijí mimo okno a soubory se načítají až tehdy, když je agent potřebuje (21).

Opakující se část kontextu — systémové pokyny a popisy nástrojů — se mezi kroky neposílá znovu, ale vrací se z mezipaměti. Model tím vidí totéž; ušetří se jen peníze a čas.

Paměť má zpravidla vlastní způsoby, jak se zkazit: zastaralé instrukce, které nikdo neupraví, spory mezi soubory na různých úrovních, příliš mnoho nepodstatného a pravidla, která se postupně posouvají, aniž by je někdo schválil. Kompakce musí přežít sama sebe — když v souhrnu zůstane jen to, co bylo napsáno naposledy, agent ztratí to, na čem stála celá práce.

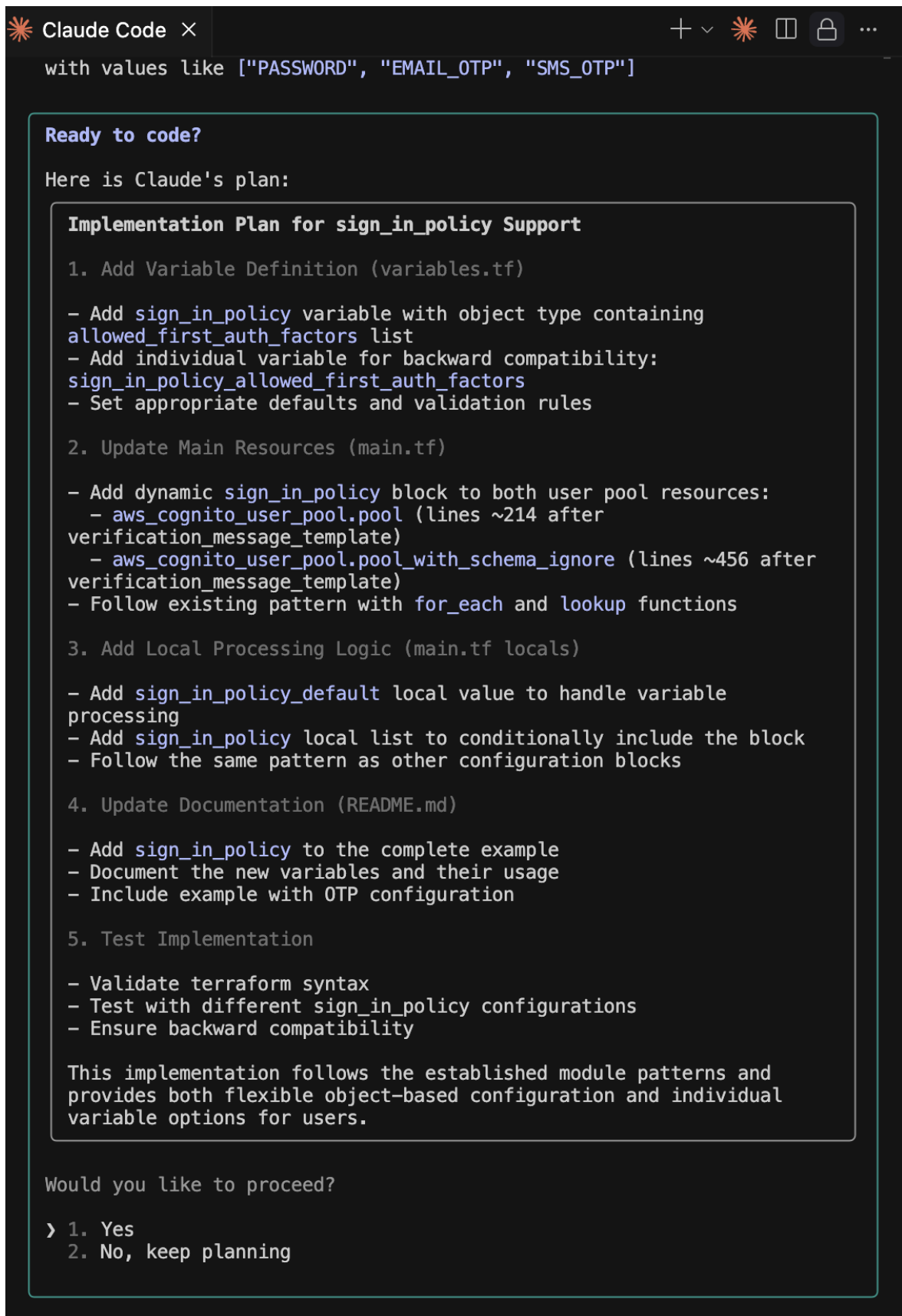
2.2 Agentické inženýrství

Agentické inženýrství (**Agentic Engineering**) označuje soubor postupů, jimiž se vývoj softwaru pomocí coding agentů stává účinným, kontrolovaným, opakovatelným a škálovatelným (22). Jeho předmětem není samotný agent ani jeho model, ale systém, v němž agent pracuje: zadání, omezení, nástroje, pravidla integrace a odpovědnost člověka. Odtud také jeho popis jako archetypu **agentického inženýra**, jehož přidanou hodnotu už netvoří psaní kódu, ale formulace zadání, řízení agentních běhů a kritické posouzení strojem vytvořených výstupů (23). Mezi hlavní oblasti patří: **Prompt engineering** (24), **Context engineering** (21), **Harness engineering** (25; 26), **Loop Engineering** (27) a **Workflow/Graph Engineering** (28). Cílem je, aby vývojář mohl efektivně a kontrolovaně delegovat dílčí úkoly agentovi, aniž by ztratil přehled o záměru, rozsahu a kvalitě výsledku.

2.2.1 Zadání a plán (Prompt & Plan)

Spolehlivé delegování práce začíná explicitním vymezením cíle, rozsahu, omezení a podmínek přijetí. Specifikace popisuje nejen požadovaný výsledek, ale také části systému, které se měnit nemají, a způsob, jakým bude výsledek ověřen. Tento přístup, označovaný jako **spec-first** nebo *spec-driven development*, dává agentovi před implementací měřitelné hranice a člověku podklad pro posouzení výsledku. Přitom jde o postup používaný i v klasickém vývoji softwaru.

V praxi to znamená, že se plán nevzniká až během práce agenta, ale před ní: jeho znění se nejprve dohodnou s člověkem, zapíše do souboru a teprve potom se agent pustí do změn.



Obrázek 7: Plán, který model navrhne, než se začne psát kód: seznam kroků podle souborů, které se mění, a otázka, zda se má pokračovat. Znění plánu je tak oddělené od jeho provádění — je to rozhodnutí, které člověk schvaluje nebo odmítne, nikoli popis práce, která už běží. Screenshot z Claude Code, převzato z (29).

Prompt engineering se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku (24). **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici (21).

2.2.2 Orchestrace a lidská integrace (Orchestration & HITL)

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdíleného stavu a integračních kontrol. Je nezbytná právě pro rozsáhlé úlohy, které se do jednoho kontextového okna nevejdou: spoléhat se tam na kompakci by znamenalo ztratit kontext, na kterém celé zadání stojí, a agent by navíc nebyl schopen pracovat na částech, které na sebe bezprostředně navazují. Čtyři vzorce na to odpovídají:

- **Coordinator/subagent** rozdělí rozsáhlou úlohu na dílčí běhy: koordinátor deleguje podúkoly specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek, nezávislé podúkoly zpracují paralelní pracovníci.

The image shows a terminal window titled "Antigravity CLI 1.0.0" with "Gemini 3.5 Flash" as the model. A colorful ASCII art logo of a rocket is on the left. The user enters a command to create three subagents for researching a data web app. The system responds with a "Thought" process, then lists the successful creation of three subagents: "ui_researcher", "data_researcher", and "code_quality_analyst". It then assigns the "ui_researcher" agent the task of researching UI/UX. The output is formatted with green and yellow highlights for subagent names and actions.

```
Antigravity CLI 1.0.0
Gemini 3.5 Flash

> can you start 3 subagents to research how to build a data web app? E.g. a UI agent,
  a data agent and a code quality analyst

▸ Thought for 2s, 852 tokens
  Initiating Project Decomposition

● DefineSubagent(Subagent "ui_researcher" defined successfully. It can now be...)
● DefineSubagent(Subagent "data_researcher" defined successfully. It can now ...)
● DefineSubagent(Subagent "code_quality_analyst" defined successfully. It can...)
  (ctrl+o to expand)

▸ Thought for 2s, 634 tokens
  Defining Subagents Concurrently

● Agent(ui_researcher: UI/UX Researcher)(Please research and provide a comprehensive
  report on building a premium-grade U...)

  I have successfully defined and launched 3 specialized subagents to research
  different aspects of building a high-quality data web application:

  1. UI/UX Researcher ( ui_researcher )
```

Obrázek 8: Rozhraní Google Antigravity CLI, tedy harnessu agenta běžícího v terminálu: koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem (30).

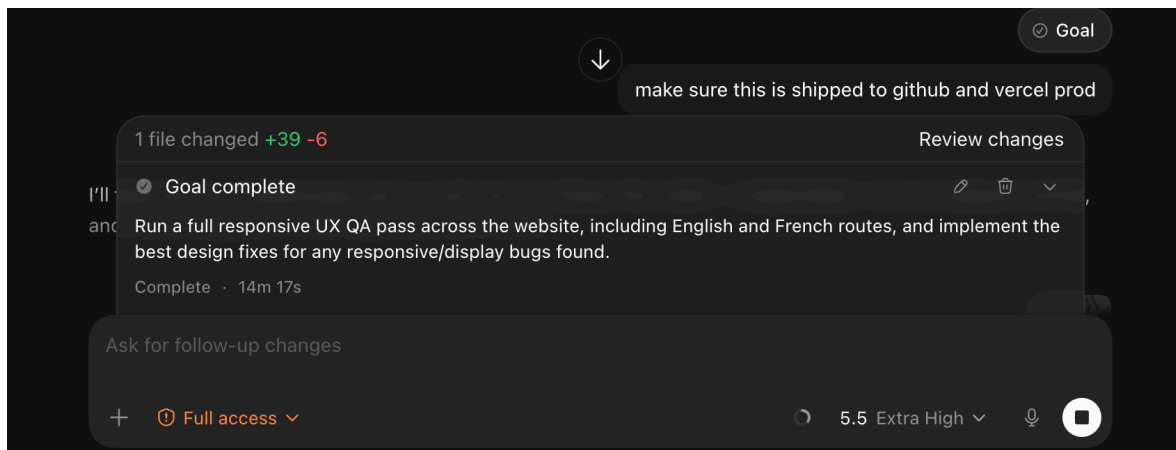
- **Workflow graph** předem určuje závislosti, pořadí a větvení fází (28). Kimi je jeden způsob, jak takový graf sestavit: místo pevného pořadí rozkládá úlohu na podproblémy až za běhu a každému dá vlastního agenta (31). Claude Code jde dál: u dynamických workflow napíše skript, který runtime vykonává na pozadí, takže smyčku, větvení i mezivýsledky drží skript místo kontextu modelu a plán je program, který lze přechíst a znovu spustit (32).

The screenshot shows the Claude Code interface with a workflow graph. On the left, under 'Phases', a list of tasks is shown with progress indicators: Foundation (8/8), AcceptanceCriteria (4/4), CodeReview (16/16, highlighted), Verify (39/39), Completeness (3/4), and Synthesis. On the right, under 'CodeReview · 16 agents', a table lists 16 agents with their respective models, token counts, tool counts, and execution times.

Agent	Model	Tokens	Tools	Time
cloud:security	claude-codex-pro	47.2k tok	15 tools	1m 32s
cloud:database	claude-deepseek-pro	141.9k tok	59 tools	7m 14s
cloud:python	Sonnet 4.6	120.4k tok	39 tools	12m 38s
cloud:stripe-deep	claude-deepseek-pro	133k tok	46 tools	5m 5s
cloud:signup-deep	claude-deepseek-pro	127.2k tok	46 tools	5m 58s
gubbi:security	claude-codex-pro	34.9k tok	17 tools	1m 14s
gubbi:database	claude-deepseek-pro	117.9k tok	73 tools	7m 36s
gubbi:mcp-deep	claude-deepseek-pro	113.1k tok	48 tools	7m 12s
gubbi:extraction-deep (...)	claude-deepseek-pro	131.4k tok	97 tools	10m 18s
gubbi:python	Sonnet 4.6	73.1k tok	27 tools	5m 19s
common:deep	claude-deepseek-pro	134.7k tok	56 tools	5m 45s
common:security	claude-codex-pro	21.5k tok	9 tools	1m 16s
testbench:deep	claude-deepseek-pro	103.8k tok	79 tools	6m 16s
infra:devops	claude-codex-pro	67.9k tok	33 tools	8m 46s
infra:security	claude-codex-pro	112.4k tok	40 tools	5m 51s
infra:architecture	claude-deepseek-pro	94k tok	42 tools	5m 57s

Obrázek 9: Postup práce dynamického workflow v Claude Code: vlevo fáze plánu se stavem, vpravo rozvinutá fáze *CodeReview* se šestnácti agenty, u každého model, spotřebované tokeny, počet nástrojů a doba běhu. Screenshot pochází z komunitního příspěvku; rozhraní i jeho chování popisuje dokumentace (32).

- **Goal loop** je nadřazená řídicí smyčka: po dílčím dokončení ReAct smyčky se pozorovaný stav porovná s cílem a podmínkami přijetí a běh skončí, nebo pokračuje další iterací. Vzor pochází z tzv. **Ralph loop**, který v roce 2025 popsal Geoffrey Huntley jako smyčku, jež opakovaně spouští agenta se stejným zadáním a stav si drží v pracovním stromu, nikoli v přepisu konverzace (33). Dnes je to běžná funkce Claude Code i Codexu (27; 34). Zásadní přitom je, že nejde o jediný pokus: agent dostává zpět to, co jeho krok vyrobil, a může podle toho postup změnit. Spolehlivost tu nevzniká z modelu, ale z toho, že se chyba stane součástí dalšího kroku.



Obrázek 10: Cíl, proti němuž agent postupuje, a jeho splnění: po dokončení dílčího kroku se stav porovná s cílem a běh v tomto případě končí, včetně doby, kterou zabral. Screenshot pochází z komunitního příspěvku; funkci popisuje dokumentace (27).

- **human-in-the-loop (HITL)** označuje bod, v němž se do automatizované smyčky doplní kontrolní brány vyžadující explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak může ponechat agentovi ohraničenou implementační práci a současně si zachovat odpovědnost za záměr a integraci.

2.3 Softwareová továrna

Softwareová továrna je systém, který přebírá co největší část práce na softwaru — od zadání přes implementaci až po revizi — a nechává člověka jen tam, kde jeho úsudek nelze nahradit (35). Pojem je o půl století starší než dnešní vlna agentů: poprvé jej použil Robert Bemer v návrhu strojově řízeného výrobního prostředí (36).

Továrna automatizovala jen kroky, jejichž postup byl předem dán. Agent vykoná i krok, jehož postup předem dán není: zadání přeformuluje, plán sestaví, chybu, kterou sám způsobí, opraví a rozhodne, na řadě je další nástroj, nebo ne. Automatizovatelná část se tím posouvá z provádění na úsudek, zatímco člověk zůstává tam, kde byl v každé dosavadní továrně — u určení, co je správné a co smí do výroby vstoupit.

PRAKTICKÁ ČÁST

DarkFactory

3 Praktická část

3.1 Metodika

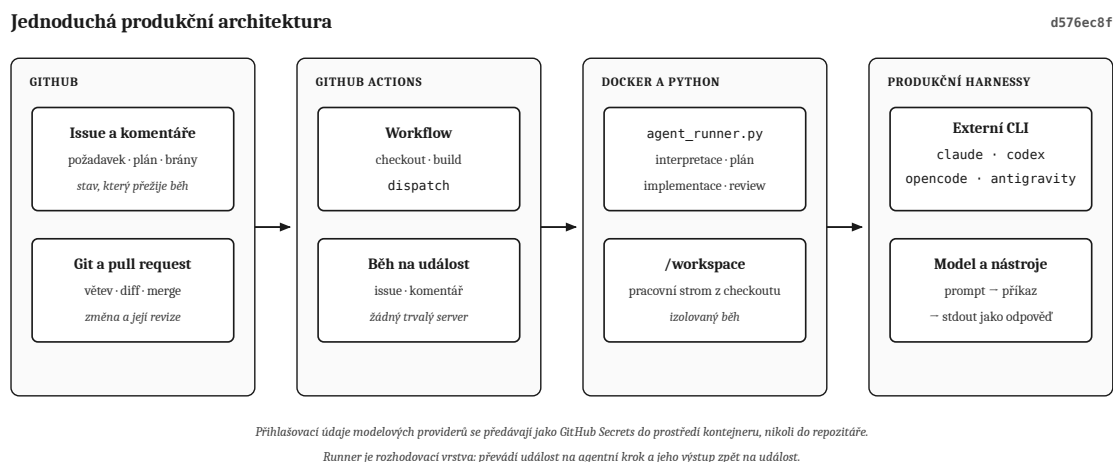
Praktická část implementuje záměrně jednoduchou produkční pipeline, v níž je vývojovým prostředím přímo GitHub a harness produkční coding agent. Cílem je ukázat, jak lze spojit události GitHubu, automatizované plánování, izolovanou práci v kontejneru a lidskou integraci do jednoho opakovatelného procesu.

Pythonovský runner, o kterém tato část pojednává, byl napsán pomocí těchž produkčních coding agentů, které následně popisuje. Cílem nebyla stavba co nejuccelenějšího systému, ale co nejmenší uzavřená smyčka, na níž lze ukázat, jak se navzájem ovlivňují agent, harness a lidský recenzent. Pozornost je proto soustředěna na tuto interakci a na její členění, nikoli na šíři nabízených funkcí.

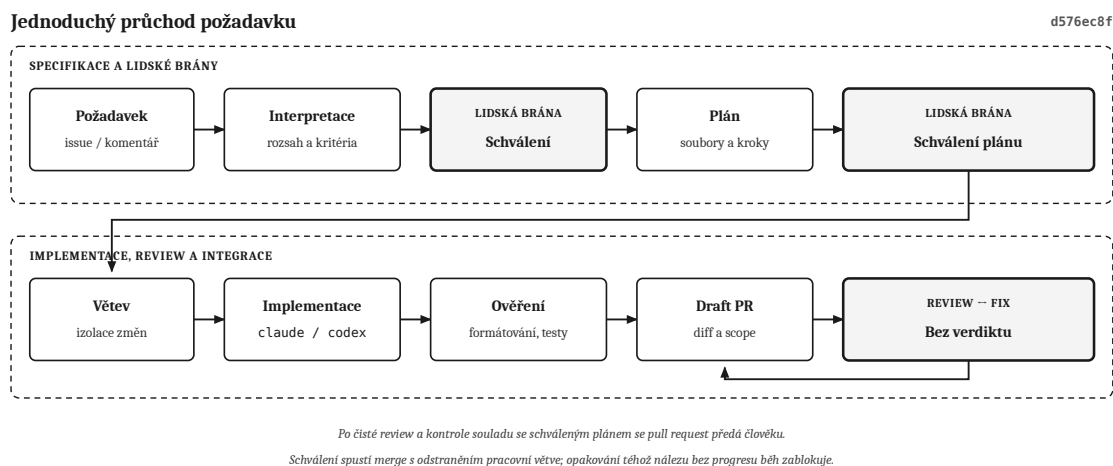
Systém bez vlastního stavového serveru, bez databáze a bez trvalého běžícího procesu je na první pohled méně schopný než plnohodnotná platforma. Oproti ní je však reprodukovatelný, dohledatelný a jehož každé rozhodnutí zůstává vidět v issue, commitu nebo průběhu kontroly. Každý nový prvek pipeline byl přidán až poté, co bylo na konkrétní úloze zřejmé, že současná podoba nestačí, takže složitost rostla postupně. Samotná pipeline se stala nástrojem, kterým byla psána i její další část.

3.2 Architektura produkčního běhu

Pro základní průchod nepotřebuje DarkFactory nic, co by muselo běžet trvale na vlastním počítači. Všechno, co pipeline potřebuje, už existuje: GitHub slouží jako rozhraní i jako trvalý stavový systém a každá práce agenta probíhá jako izolovaný běh v GitHub Actions. Členění do čtyř vrstev znázorňuje Obrázek 11 a pořadí jednotlivých kroků Obrázek 12.



Obrázek 11: Architektura: GitHub poskytuje události a vývojový stav, GitHub Actions výpočet, Docker odděluje běh a pythonovský runner převádí událost na agentní krok. Produkční harnessy zajišťují model, nástroje a pozorování; registr jich má osm, obrázek ukazuje čtyři (8).



Obrázek 12: Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka se opakuje, dokud nálezní nezmizí (8).

Workflow `agent.yml` reaguje na otevření issue, nový komentář, review komentář, `repository_dispatch` nebo ruční spuštění. Podmínka na úrovni jobu ověřuje, zda je agent pro repozitář povolen, a filtruje automatické komentáře, aby vlastní výstup pipeline nevytvářel nové události. Workflow pak provede checkout cílového repozitáře, sestaví obraz podle `docker/Dockerfile.agent` a spustí příkaz `dispatch` v kontejneru. Obraz je sestaven

z jedné definice, takže každý spotřebitel běží stejný runner, zatímco repozitář, na němž se pracuje, zůstává checkoutem volajícího.

pythonovský runner není náhradou harnessu. Je rozhodovací a integrační vrstvou, která převádí událost GitHubu na konkrétní agentní krok, zpracovává jeho výstup a vyvolává další událost. Modelové kroky jsou přitom stále prováděny harnessem nad explicitně předaným pracovním stromem (8).

Každý harness je tak definován deklarativně: binár, způsob, jak se z promptu sestaví příkazová řádka, a způsob přihlášení. Přidání harnessu je tedy změna dat, nikoli kódu (8). V popsané revizi je v registru osm harnessů a pořadí, v němž se zkoušejí, je konfigurovatelné; harness, jehož binár v obrazu chybí, se přeskočí, takže jediný obraz unese jen podmnožinu nástrojů, která je v něm nainstalována. Tvrzení, že struktura procesu nezávisí na volbě nástroje, je proto v této revizi doslova pravdivé: strukturu popisuje registr, ne kód.

Přihlašovací údaje se neukládají do repozitáře. GitHub App nebo jiný autorizovaný token se používá pro checkout, issue, pull requesty a push; přihlašovací údaje modelových providerů jsou předány workflow jako GitHub Secrets a následně prostředím kontejneru. Tím pipeline odděluje své automatizační oprávnění od přihlašovacího materiálu agenta a umožňuje změnit poskytovatele bez změny pracovního stromu (8).

3.3 Interpretace a plánování požadavku

Výchozím bodem je issue s formulovaným požadavkem. Požadavek se však nevytváří jako volný text: repozitář nabízí předlohu, která vyžaduje vyplnit několik polí, a právě ta určují, co pipeline dostane.

Pole předlohy issue

Jak to vypadá v issue 727

Verbatim User Request povinné

restore the agent container build from the checked-in
Bun lock owner

Area / Component povinné

area:ci

Request Type povinné

bug

místo vyplnění předlohy — co napsal požadavek 727

Problem — `Dockerfile.agent` kopíruje kořenový `bun.lock`, ale repozitář má jen `harness/bun.lock`; běh agenta tak padá ještě před spuštěním. Viz `### Contract` v issue.

Tabulka 1: Předloha požadavku a nejstarší požadavek projektu, issue 727 z 20. září 2026. Předloha má šest polí, z nichž tři povinná, ale požadavek, který pipeline zpracovala jako první, předlohu nevyplnil: napsal volně, pod nadpisy `Problem` a `Contract`. Runner předává modelu titulek i tělo issue, takže právě to rozhoduje, co se do zadání dostane (8).

Příklad je záměrně malý a měřitelný: rozsah se vejde do jednoho kontextového okna a podmínky dokončení lze přepsat na testy. To nejsou náročky pipeline, ale podmínky, za kterých je nasazení vůbec smysluplné — a právě proto je předloha součástí popisu. Je tu ale i věc, kterou předloha neumí: přijaté akceptační podmínky jsou text, který vstupuje do kontextu, a nikdo je v popsané revizi nepřevádí na spustitelnou kontrolu. Podmínka, kterou nelze spustit, se při prokazování chová stejně jako podmínka, která neplatí.

Runner načte jeho titulek a text, vyžádá si od modelu interpretaci a zapíše výsledek jako komentář. Komentář má oddělovat doslovné shrnutí požadavku, architektonický rozsah a návrh verifikace. Tím se z chatové odpovědi stane zkontrolovatelný návrh, který lze před dalším během přijmout nebo opravit. Na rozdíl od prostého chatu je tu interpretace artefakt, který přežije běh a dá se k němu vrátit.

Po schválení interpretace je workflow spuštěno znovu a runner založí *child* issue s názvem začínajícím na `Plan:`, nativně propojené s původním issue. Model dostane schválený požadavek a sestaví implementační plán s očekávanými změnami, soubory nebo oblastmi repozitáře a kroky ověření. Plán se opět zobrazí v issue a schvaluje se samostatně. Schválení je tak explicitní bránou: samotná schopnost agenta plán vytvořit neznamena oprávnění

měnit kód. Obě brány před vznikem větve jsou lidské, a protože jsou oddělené, lze schválit porozumění zadání a odmítnout plán, který z něj vychází.

Zpětná vazba člověka není součástí nového vývoje od začátku. Komentář se změnou nebo odmítnutím se předá zpět interpretaci nebo plánování, takže se opravuje rozhodnutí před vytvořením pracovní větve. Tento jednoduchý model odděluje porozumění zadání, plánování a vlastní implementaci bez potřeby složitého grafového orchestrátoru (8), což je volba, již literatura připouští jako legitimní: paralelní práce je užitečná tam, kde na sobě úlohy nezávisí, a jinde je to jen zdroj sporu (28).

3.4 Implementace a automatická revize

Po schválení plánu runner vytvoří nebo načte pracovní větev odvozenou z výchozí větve. Implementační instrukce obsahuje schválený plán, omezení na jeho rozsah a pravidla pro testy a dokumentaci. Harness následně může procházet repozitář, upravovat soubory a používat nástroje nad `/workspace`; změny zůstávají izolované mimo výchozí větev.

Po implementaci runner spustí dostupné formátovací nástroje a deklarované testovací sady. Při neúspěchu předá výstup kontroly agentnímu kroku `fix`, který má opravit chybu bez opuštění schváleného rozsahu. Testovací sady se přitom spouštějí pouze jednou a po opravě už ne: kód se zformátuje, zakomituje a odešle dál, i kdyby opravená verze stále neprocházela. Opakovatelná kontrola tedy rozliší dobrý a špatný stav, ale není překážkou — změna s neprocházejícími testy se dostane do draft pull requestu stejně jako změna procházející. Následně runner vytvoří commit, odešle větev a otevře draft pull request, čímž se změna oddělí od svého posouzení: implementace může být dokončena, ale pull request ještě není připraven k merge (8).

Na draft pull requestu začíná automatická review smyčka. Každá iterace načte aktuální větev a diff a předá je modelové revizi, zatímco průběžná integrace znovu spustí tytéž nástroje. Nalezený problém runner zveřejní a spustí nový běh s fází opravy, která změní větev a spustí další review. Nález vyžadující zásah mimo plán se neodmítne: runner jej zapíše jako Plan Deviation s odůvodněním na původním issue a plán tím doplní. Schválený plán tedy není neměnný, ale každá jeho změna je zapsaná a odůvodněná. Smyčka však může skončit bez jakéhokoli verdiktu a bez komentáře, pull request zůstane ve stavu draft a nikdo zvenčí nepozná, zda review prošlo nebo se jen zastavilo. Jediným stavem, který v této revizi znamená zablokování, je vyčerpaná kvóta — tehdy se zapíše kontrolní bod, stav na projektové tabuli a komentář s výzvou k obnovení (8).

Tabulka 1 ukazuje požadavek podaný právě kvůli této meze: žádá, aby se běh po opakováních stejného nálezu *zastavil a napsal to výslovně do issue*. Popsaná revize to neumí, ačkoli je podmínka zastavení v té dokumentaci uvedena výslovně (27).

Po čisté review ještě proběhne kontrola souladu výsledného diffu se schváleným plánem. Ani ta není porovnáním sad souborů: je to druhý dotaz modelu, tentokrát na shodu s textem plánu. Teprve když oba dotazy vyjdou bez nálezu, je draft pull request označen jako připravený k lidské revizi. Je důležité, že automatická revize může skončit bez nálezu, aniž by zaručila, že implementace odpovídá zadání, a posuzuje přitom plán, jehož text napsal tentýž model. Dodavatelé tento typ hodnotitele popisují stejně omezeně (34), a je proto s první branou rovnocenný, ne její náhradou.

3.5 Zpětná vazba, schválení a úklid

Po otevření pro lidskou revizi může člověk použít schvalovací workflow nebo zpětnou vazbu na pull requestu. Změnový požadavek zachycuje runner jako opravný běh na stejné větvi. Agent obdrží plán, konkrétní zpětnou vazbu a aktuální kontext větve, provede úpravu, uloží ji jako commit a znovu spustí automatickou review smyčku. Tím se zpětná vazba nevrací do schváleného plánu ani nevyžaduje nový počátek celého požadavku; zachovává se pouze řetězec revizí na jednom pull requestu (8).

Schválení pull requestu zpracovává samostatný workflow. Ten nejprve ověří, kdo akci provedl, a ověření je užší, než by se z krátkého popisu zdálo: aktér se porovnává se seznamem dvou pevně zapsaných účtů, vlastního repozitáře a jednoho dalšího. Neověřuje se tedy autor issue ani úroveň oprávnění, ale členství v tomto seznamu (8). Teprve pak workflow převede draft na ready stav, zkontroluje požadované schválení a zapne merge; při úspěchu použije `--delete-branch`, takže se pracovní větev po sloučení odstraní. Integrace tedy nekončí pouhým otevřením pull requestu: zahrnuje přechod do ready, splnění ochrany větve, merge a bezpečné odstranění větve (8; 37).

Je tu třeba přiznat věc, která třetí kritérium metodické části oslabuje. Jestliže ochrana větve stále vyžaduje schválení, workflow podá *zástupné* schválení jménem jiného účtu, aby merge neuvázl (8). Podmínkou je druhý přihlašovací údaj; kde není, pipeline čeká na skutečné schválení člověka. V konfiguraci, v níž je nastaven, je však schválení, které by mělo být lidským rozhodnutím, vydáno strojem. Třetí kritérium je proto plněno jako *bod*, na kterém člověk rozhodne, a ne jako *záruka*, že rozhodne on; rozdíl je malý v textu a velký v tom, co z něj plyne, a práce ho záměrně nerozšiřuje.

Tato architektura je záměrně jednoduchá. GitHub poskytuje události, schválení, uložení změn a průběh kontroly, GitHub Actions výpočet, Docker odděluje běh, Python koordinuje a harness vykonává agentní práci. Složitější služby, trvalý stavový server a více souběžných agentů jsou záměrně mimo základní návrh. Výhodou je snadná reprodukovatelnost a viditelnost každého rozhodnutí; nevýhodou je zároveň závislost na dostupnosti GitHubu a na kvalitě promptu, modelu a pravidel repozitáře, kterou samotná automatizace neodstraňuje.

Směr vývoje, který práce záměrně nepopisuje, je vymezen v Kapitola 1.2. Zde je tedy přesnější dodat, kde popsán průchod končí: je popsán stav, v němž pythonovský runner volal cizí produkční CLI (8). Popsaná architektura je tudíž mezníkem, nikoli vyvrcholením, a její popis nenahrazuje posouzení této meze, které patří do Kapitola 4.2. Projekt (38) se od tohoto stavu posunul dál; kam se posunul a proč, je předmětem práce následující, a zde to záměrně není rozvedeno.

4 Závěr

4.1 Zjištění

První zjištění se týká volby vývojového prostředí. Požadavek, jeho schválení, plán, zpětná vazba i výsledná revize zůstávají v GitHubu, takže jednotlivé běhy nemusí sdílet vlastní databázi ani trvalý proces, jak popisuje Kapitola 3.2. Plán je samostatné issue propojené s původní žádostí, takže se k němu dá vrátit a porovnat s výsledným diffem (8).

Druhé zjištění potvrzuje oddělení lidského rozhodnutí od modelového návrhu. Issue nejprve vyvolá interpretaci, následně plán a teprve po schválení obou kroků se vytvoří pracovní větve a spustí implementace. Tím pipeline nevytváří změny v hlavní větvi pouze na základě samotného návrhu agenta (8).

Třetí zjištění se týká průběhu revize a jeho pořadí. Implementace je nejprve odevzdána jako draft pull request. Automatická kontrola nejprve předá diff modelové review a teprve poté, jakmile je čisté, porovná změněné soubory se schváleným plánem. Nalezené problémy spouštějí samostatný běh opravy, po němž následuje nová kontrola. Teprve čistá review a kontrola souladu s plánem otevrou pull request pro lidskou revizi (8).

Čtvrté zjištění se týká zpětné vazby a integrace. Změnový požadavek na pull requestu spouští opravu na stejné větvi a po pushi znovu vstupuje do review smyčky. Schválení pull requestu naopak spouští merge s odstraněním větve, takže pipeline nekončí automatickým vytvořením kódu, ale přechází do explicitního lidského schválení a následné integrace. Kdo schválení vykoná, je v této revizi určeno jen dvojčlenným seznamem účtů, a je-li ochrana větve stále žádající, může potřebné schválení doplnit workflow jménem jiného účtu (8).

Páté zjištění je záporné a práce ho ponechává. Tři předpoklady, na nichž popis průběhu stál, v této revizi neplatí. Revize se může zastavit bez zaznamenaného verdiktu, takže nerozliší průchod od zastavení; zablokovaný stav zde neznamena opakovaný nález, ale vyčerpanou kvótu; a testy se před sloučením nespouštějí znovou, takže změna, která neprošla, na pull request dorazí. Podmínka zastavení, která první z těchto mezer vyřešila, je v dokumentaci cílové smyčky popsána výslovně (27); její vyplnění je věcí následné práce, ale zjištění, že chybí, do výsledků patří (8).

Na výzkumnou otázku odpovídají tři věci. První je stav, který leží mimo model, takže se k němu lze vrátit bez dotazu na model. Druhý je druh rozhodnutí. Brány před vznikem větve jsou lidské a brány před sloučením jsou obě dotazem na model. Kapitola 4.2 upozorňuje, že proto nejsou navzájem nezávislé. Třetí je okamžik, v němž rozhodne účet ze seznamu dvou pevně zapsaných; jak se to zaznamená, popisuje čtvrté zjištění.

4.2 Diskuse

Zjištění lze interpretovat jako konkrétní podobu agentického inženýrství: praktická autonomie není vlastností samotného modelu, ale výsledkem návrhu prostředí, v němž model pracuje. GitHub poskytuje trvalý kontext a lidskou odpovědnost, Python určuje přechody mezi kroky, Docker omezuje běhové prostředí a harness propojuje model s nástroji a pozorováními (8). Tato dělba odpovědnosti odpovídá tomu, jak obor takové systémy popisuje (25; 39).

Review smyčka ukazuje výhodu a zároveň omezení průběžné kontroly. Testy a formátovací nástroje poskytují zpětnou vazbu, kterou lze opakovat, ale ani druhá brána — kontrola souladu s plánem — deterministická není: jde o druhý dotaz modelu, a je tedy stejně pravděpodobnostní jako review, které jí předchází. Opakované iterace zvyšují počet příležitostí k nálezům, nikoli záruku konečné správnosti. Zastavení po vyčerpání iterací je ochranou před nekonečným během, ale bez zaznamenaného verdiktu není ani touto ochranou úplně: kdo se později ptá na stav, nemá v issue co číst (8).

Záporné zjištění ukázalo i druhou věc. Opakovatelná kontrola v této revizi rozliší dobrý a špatný stav, ale není překážkou: testy běží jednou a po opravě už ne, takže změna, která neprošla, se do pull requestu dostane stejně jako změna, která prošla. Rozlišení mezi kontrolou, která brání, a kontrolou, která pouze hlásí, je tedy neúplné, a je to vlastnost konfigurace, nikoli principu. Dodavatelé, kteří takovou bránu prodávají, popisují její omezení shodně: hodnotitel nevolá nástroje a posuzuje jen to, co pracující model sám ukázal (34). Samoshodnocení agentů selhává určeným směrem (25). Rozdělení práce od hodnocení tedy shovívavost neodstraňuje samo o sobě; odstraňuje ji až to, že kontrolu, kterou by bylo možné zopakovat, zajišťuje návratový kód místo názoru.

Jednoduchost návrhu má také ekonomickou a epistemickou cenu. Nevyžaduje žádnou vlastní infrastrukturu a celý průběh je dohledatelný v issue, commitech, kontrolách a pull requestu. Na druhé straně pipeline závisí na dostupnosti GitHubu, kvalitě issue, schopnosti modelu pracovat s nástroji a pravidlech, která definují akceptační podmínky. Jednoduchá architektura tedy usnadňuje reprodukci postupu, ale nenahrazuje odpovědnost člověka za architekturu, bezpečnost a přijetí výsledku (8).

Opakovatelná a pravděpodobnostní pozorování se proto doplňují, ale nenahrazují. Formátovací nástroje a testy poskytují pozorování, které se dá opakovat; review, kontrola souladu s plánem, plánování a opravy interpretují zadání a neočekávané nálezy. Z toho plyne, že obě brány před sloučením nejsou nezávislé: jsou to dva dotazy téhož druhu, a druhý z nich navíc vychází z plánu, jehož text sám napsal model. Tato kombinace odpovídá cíli agentického

inženýrství zvýšit užitečnost delegované práce při zachování explicitních bran, ale její skutečná kvalita zůstává závislá na datech, promptu, modelu a konkrétním repozitáři (8).

Praktická část má omezení, která je třeba pojmenovat, protože se dotýkají platnosti závěrů. Spočívají ve třech vrstvách.

První je vlastnictví. Agentní smyčku v popsané realizaci autor nenavrhuje: sestavuje ji cizí harness, která spravuje kontext, volá nástroje a vyřizuje oprávnění. Co autor navrhuje, je vrstva nad ní, tedy runner, který rozhoduje, kdy se smyčka spustí, co smí agent změnit a kde práce skončí. Odpověď na výzkumnou otázku je tím užší, než kdyby smyčka byla implementována od začátku, a je takto i formulována.

Druhé je rozsah. Zkoumaný stav je záměrně jen počáteční: runner volá cizí harnessy a všechny agentní kroky vede přes ně. Jde tedy o míru autonomie dosažitelnou touto konkrétní konfigurací, nikoli o míru dosažitelnou s vlastní smyčkou, a tím záměrně neřeší otázku, kterou řeší práce následující. Tvzení práce se proto vztahuje k této konfiguraci a je opřeno o konkrétní revizi (8).

Třetí je složení vrstev. Popsaný systém má nad sebou dvě vrstvy harnessu: vnější runner s branami a uloženým stavem, vnitřní produkční CLI, které modelové kroky vykonává. Během jednoho běhu se tedy model setkává se dvěma sadami pravidel současně, a to, kterému pravidlu se podřizuje a co vnější vrstva stojí, zůstává nezodpovězené. Část odpovědi je přesto veřejně dostupná: dokumentace vrstvení běhů rozlišuje předání práce, při němž zůstává jediný cyklus, od volání agenta jako nástroje, při němž vzniká vnořený běh s vlastním cyklem, limitem iterací a žádostmi o schválení. Neurčuje však výši nákladu — „the nested turns do not increment the outer run’s turn counter“ (26) — takže účetnictví vnější vrstvy práci vnořeného běhu nevidí. Zda je takový náklad v popsaném systému významný, zjištěno nebylo.

Z prvního omezení plyne i obrana závěrů. Harness je v popsané architektuře zaměnitelnou závislostí, stejně jako databáze nebo runner. Co práce především ukazuje, není volba konkrétního CLI, ale struktura procesu: že požadavek musí projít interpretací, plánem a dvěma lidskými branami dříve, než vznikne větev, a že změna se přebírá jako konkrétní revize na jednom pull requestu, dokud nesplní obě brány. Tato struktura je vůči tomu, kdo vykonává modelové kroky, invariantní. Právě to je obsahem agentického inženýrství ve smyslu vymezeném v teoretické části: jde o vlastnost této práce, neboť popsaná struktura je záměrem autora a nezávisí na tom, kdo vykonává modelové kroky. To však není obecná zákonitost, a závěr práce proto přenositelnost struktury na libovolný nástroj neprokazuje; naopak, tam, kde se konvence přenášejí z jednoho typu běhu do druhého, je výsledek opačný (40).

Hypotéza se potvrzuje: strukturu, kterou práci řídí, autor navrhuje sám, zatímco úsudek o tom, co je v jednotlivém kroku správné, přebírá od smyčky, kterou nevlastní.

4.3 Shrnutí

Jazykový model sám o sobě pouze generuje výstup v závislosti na vstupním kontextu. Agent z něj vzniká až propojením s nástroji, prostředím, stavem a pozorováním, které zajišťuje harness (15; 25). Ani toto propojení však pro účinné nasazení ve vývoji softwaru nestačí bez postupů agentického inženýrství, které vymezují rozsah autonomie, určují body lidského rozhodnutí a zajišťují ověřování výsledků.

Implementace pipeline DarkFactory ukázala jednoduchou produkční realizaci těchto postupů. GitHub slouží jako vývojové prostředí a místo trvalého stavu, GitHub Actions jako výpočetní prostředí, Dockerový kontejner jako izolace a produkční harness pro modelovou práci s nástroji. Požadavek prochází interpretací a plánováním, implementace probíhá na samostatné větvi a změna je předána přes draft pull request a opakovací smyčku review–fix. Po čisté review a kontrole souladu s plánem následuje lidské schválení, merge a odstranění větve.

Úloha vývojáře se v tomto uspořádání posouvá od rutinního provádění kódu k formulaci zadání, schvalování plánu, definování akceptačních podmínek a rozhodování, zda je výsledek připraven k integraci. Práce tedy nepopisuje plnou autonomii agenta, ale kontrolované předání vybrané části softwarového inženýrství.

Zjištění zároveň ukazují meze této realizace. Modelové review může být opakované, ale není deterministickou zárukou správnosti. Review smyčka se může zastavit bez zaznamenaného verdiktu, takže průchod od zastavení rozlišit není; zablokovaný stav zde znamená vyčerpanou kvótu, nikoli opakovaný nález; a opakovatelná kontrola před sloučením běží, ale změně s neprocházejícími testy zabrání jen tehdy, když ji zopakuje, což popsaná revize nečiní.

Cíle práce se podařilo naplnit v rozsahu, který si sama vymezila. Hypotéza se potvrdila: praktická autonomie je vlastností návrhu systému, který práci řídí, a nikoli vlastností modelu, který v něm pracuje. Strukturu, která práce popisuje, autor navrhuje sám; úsudek o tom, co je v jednotlivém kroku správné, však přebírá od smyčky, kterou nevlastní, a právě v tomto rozdílu leží hranice, za kterou by pomohlo vlastnit i tuto smyčku.

Pro další výzkum z toho vyplývají tři směry. První je vlastní harness: popsat architekturu, kterou si tato práce záměrně neřešila, a porovnat její výsledky s výsledky cizích nástrojů na stejných úlohách. Druhý je měření: dosavadní zjištění jsou kvalitativní, chybí jim to, co by umožnilo říct, o kolik se cyklus zkrátil a kde přesně review smyčka nachází nejvíce nálezu, a prvním krokem je nechat rozlišit čistou recenzi od zastavené smyčky a nechat opakovanou kontrolu běžet znovu, protože dnes hlásí, ale nebrání. Třetí je otázka, která zůstala v diskusi otevřená: když vnější vrstva je sama o sobě harnessem, setkává se model během jednoho

běhu se dvěma soubory konvencí, a práce neprovedla měření, které by ukázalo, kterých následuje a co vnější vrstva stojí.

V praxi má práce přímý dopad v podobě jednoduchého pravidla: delegovat agentovi až to, co lze zkontrolovat, a ponechat mu vlastní smyčku jen tam, kde je výhodná. Kdo takovou pipeline staví, nemusí začínat vlastním harnessem; začíná stavem uloženým mimo model, dvěma branami před vznikem větve a oddělenou kontrolou výstupu. A ještě jedno, které z popsaného systému vyplynulo a které je stejně důležité jako tyto tři: je-li brána, která má být lidská, v jedné konfiguraci schopna vyplnit stroj, pak její jméno neznamena nic, pokud u ní není zapsáno, kdo ji vykonává.

Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. GITHUB. *Asking GitHub Copilot questions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide>
3. KHURANA, Sonam a Anusha VADDI. ChatGPT From the Perspective of an Academic Oral and Maxillofacial Radiologist. *Cureus* [online]. 2023, **15**(6). Dostupné z: doi:10.7759/cureus.40053. Obrázek 4. Screenshot z bezplatné verze ChatGPT, převzato jako ilustrace
4. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
5. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-codex/>
6. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-the-codex-app/>
7. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://www.gradually.ai/en/how-many-people-use-ai/>. Published 2026-08-23
8. MARIUS, Patrik. *DarkFactory revize d576ec8f* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://github.com/marius-patrik/DarkFactory/tree/d576ec8fdb78a654bffc9d0da165aa81a97b673>. Poslední revize, v níž ještě pipeline volá produkční harneysy
9. MERRIAM-WEBSTER. *Agentic: Definition & Meaning* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://www.merriam-webster.com/dictionary/agentic>. Původ slova: agent + -ic, poprvé doloženo 1966
10. TRIVEDI, Vivek. *The Anatomy of an Agent Harness* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>. LangChain, 10. března 2026
11. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need. *ar-*

- Xiv preprint arXiv:1706.03762* [online]. 2017. Dostupné z: <https://arxiv.org/abs/1706.03762>
12. BROWN, Tom B., Benjamin MANN, Nick RYDER, Melanie SUBBIAH, Jared KAPLAN, Prafulla DHARIWAL, Arvind NEELAKANTAN, Pranav SHYAM, Girish SASTRY, Amanda ASKELL a OTHERS. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* [online]. 2020, **33**. Dostupné z: <https://arxiv.org/abs/2005.14165>
 13. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>
 14. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: <https://arxiv.org/abs/2210.03629>
 15. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: <https://www.anthropic.com/engineering/building-effective-agents>
 16. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://agentskills.io/home>
 17. OPENAI. *Lifecycle | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
 18. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://modelcontextprotocol.io/specification/2026-07-28/server>
 19. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://agents.md/>
 20. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: <https://arxiv.org/abs/2307.03172>

21. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
22. WILLISON, Simon. *What is agentic engineering?* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://simonwillison.net/guides/agentic-engineering-patterns/what-is-agentic-engineering>. Kapitola průvodce Agentic Engineering Patterns
23. ALENEZI, Mamdouh. Educating the Agentic Engineer: Curricula, Collaboration, and Continuous Learning in the AI Era. *arXiv preprint arXiv:2607.29610* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://arxiv.org/abs/2607.29610>
24. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>
25. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
26. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
27. OPENAI. *Using Goals in Codex: Persistent Objectives for Long-Running Work* [online]. 2026 [vid. 2026-09-27]. Dostupné z: https://developers.openai.com/cookbook/examples/codex/using_goals_in_codex
28. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://openai.github.io/openai-agents-python/multi_agent/
29. GALLARDO, Luis. *Beyond Cursor: My Experience with Claude Code and Sequential Thinking* [online]. 2025 [vid. 2026-09-28]. Dostupné z: <https://lgallardo.com/2025/07/09/beyond-cursor-claude-code-sequential-thinking/>. Obrázek s plánem, který Claude Code předkládá ke schválení. Převzatá ilustrace rozhraní, nikoli záznam běhu popsané pipeline
30. GOOGLE. *Antigravity CLI* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://antigravity.google/product/antigravity-cli>. Sekce Subagents functionalities. Převzatá ilustrace rozhraní, nikoli záznam běhu
31. KIMI TEAM. Kimi K2.5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://arxiv.org/abs/2602.02276>

32. ANTHROPIC. *Orchestrate subagents at scale with dynamic workflows* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://code.claude.com/docs/en/workflows>. Sekce Dynamic workflows. Převzatá ilustrace rozhraní, nikoli záznam běhu
33. HUNTLEY, Geoffrey. *Ralph Wiggum as a „software engineer“* [online]. 2025 [vid. 2026-09-27]. Dostupné z: <https://ghuntley.com/ralph/>. 14. července 2025. Původní text, který techniku pojmenoval
34. ANTHROPIC. *Keep Claude working toward a goal* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://code.claude.com/docs/en/goal>
35. NATO. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968* [online]. Brussels: NATO, Scientific Affairs Division, 1969 [vid. 2026-09-27]. Dostupné z: <http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF>. 231 s. Zpráva o softwarové krizi; stažitelný přepis z roku 2001 má 136 stran, číslování citací se vztahuje k původnímu vydání
36. COMPUTER HISTORY MUSEUM. *Robert W. Bemer* [online]. [vid. 2026-09-27]. Dostupné z: <https://history.computer.org/bemer.html>. Životopis uvádí vynálezce pojmu software factory a jeho působení v General Electric
37. GITHUB. *About pull requests / GitHub Docs* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
38. MARIUS, Patrik. *DarkFactory* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://github.com/marius-patrik/DarkFactory>. Repoitář projektu, na němž praktická část staví
39. ANTHROPIC. *Scaling Managed Agents: Decoupling the brain from the hands* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/managed-agents>
40. THANGARAJAH, Kishanthan, Boyuan CHEN a Ahmed E. HASSAN. DCAS: Decoupling CLI Agent Scaffolding to Internalize Planning across Scaffolds. *arXiv preprint arXiv:2608.06113* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://arxiv.org/abs/2608.06113>. 6. srpna 2026. Předběžný výtisk; patová hlavička PDF nese ASE 26, záznam na arXiv trefu neuvádí

Seznam součástí textu

Obrázek 1	Doplňování kódu přímo v editoru: model navrhuje pokračování řádku, které člověk přijme nebo odmítne (1).....	5
Obrázek 2	Chatbot odpovídá textem a práci nechává člověku: místo aby snímek posoudil, přizná, že obrázky neumí, a je to uživatel, kdo musí dojít k závěru. Screenshot z bezplatné verze ChatGPT, převzato z (3).....	6
Obrázek 3	Odhad rozdělení uživatelů generativní AI podle typu (7).....	7
Obrázek 4	Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov <i>král</i> , <i>královna</i> , <i>muž</i> a <i>žena</i> (13).....	10
Obrázek 5	Vzor ReAct: myšlenka, akce a pozorování se střídají a každý z nich se připojí do kontextu, takže smyčka pokračuje na tom, co už model viděl. Končí odpovědí místo dalšího požadavku na nástroj (14).....	11
Obrázek 6	Kontextové okno v Claude Code během jednoho běhu: systémový prompt, nástroje a zadání zůstávají, zatímco myšlenky, volání nástrojů a jejich výsledky se vrší až k hranici okna. Po kompakci zůstane souhrn rozhodnutí, poslední krok a pět naposledy otevřených souborů. Vedle toho běží tři věci: starší výsledky nástrojů se čistí, poznámky žijí mimo okno a soubory se načítají až tehdy, když je agent potřebuje (21).....	12
Obrázek 7	Plán, který model navrhne, než se začne psát kód: seznam kroků podle souborů, které se mění, a otázka, zda se má pokračovat. Znění plánu je tak oddělené od jeho provádění — je to rozhodnutí, které člověk schvaluje nebo odmítne, nikoli popis práce, která už běží. Screenshot z Claude Code, převzato z (29).....	15
Obrázek 8	Rozhraní Google Antigravity CLI, tedy harnessu agenta běžícího v terminálu: koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem (30).....	16
Obrázek 9	Postup práce dynamického workflow v Claude Code: vlevo fáze plánu se stavem, vpravo rozvinutá fáze <i>CodeReview</i> se šestnácti agenty, u každého model, spotřebované tokeny, počet nástrojů a doba běhu. Screenshot pochází z komunitního příspěvku; rozhraní i jeho chování popisuje dokumentace (32).....	17
Obrázek 10	Cíl, proti němuž agent postupuje, a jeho splnění: po dokončení dílčího kroku se stav porovná s cílem a běh v tomto případě končí, včetně doby, kterou zabral. Screenshot pochází z komunitního příspěvku; funkci popisuje dokumentace (27).....	18
Obrázek 11	Architektura: GitHub poskytuje události a vývojový stav, GitHub Actions výpočet, Docker odděluje běh a pythonovský runner převádí událost na agentní	

	krok. Produkční harnessy zajišťují model, nástroje a pozorování; registr jich má osm, obrázek ukazuje čtyři (8).....	22
Obrázek 12	Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka se opakuje, dokud nález nezmizí (8).....	22
Tabulka 1	Předloha požadavku a nejstarší požadavek projektu, issue 727 z 20. září 2026. Předloha má šest polí, z nichž tři povinná, ale požadavek, který pipeline zpracovala jako první, předlohu nevyplnil: napsal volně, pod nadpisy <code>Problem</code> a <code>Contract</code> . Runner předává modelu titulek i tělo issue, takže právě to rozhoduje, co se do zadání dostane (8).....	24